Computação Paralela e Distribuída

Atividade Prática Avaliativa 01 - ED 03

Chat TCP com servidor e múltiplos clientes
Chat UDP com grupo multicast

Relatório técnico, instruções de execução e respostas às questões teóricas.

Este projeto implementa um chat em Java de duas formas: uma versão cliente-servidor com sockets TCP e uma versão sem servidor central que usa UDP multicast. O exercício demonstra comunicação entre processos, atendimento simultâneo por threads e as diferenças práticas entre TCP e UDP.

Escopo e entregáveis

- `ServidorTCP.java`: recebe clientes na porta TCP 12345, cria uma thread por conexão e retransmite mensagens a todos os participantes.
- `ClienteTCP.java`: solicita apelido, envia mensagens e recebe as mensagens do servidor em uma thread separada.
- `ClienteMulticastUDP.java`: envia e recebe datagramas no grupo multicast `230.0.0.1`, porta UDP 12346.
- `RESPOSTAS.md`: respostas teóricas e registro da verificação prática.
- `Relatorio_Atividade_Pratica_Avaliativa_01.pdf`: relatório para entrega junto ao código.

O comando `/sair` encerra o cliente TCP ou UDP. Não há interface gráfica, persistência de mensagens nem dependências externas.

Funcionamento

Chat TCP

1. O servidor abre a porta 12345 e espera conexões.
2. Cada cliente estabelece uma conexão e envia seu apelido como primeira linha.
3. O servidor cria uma thread para cada cliente. Essa thread lê as mensagens recebidas e as repassa a todos os clientes conectados, inclusive ao remetente.
4. A thread de leitura do cliente mostra as mensagens recebidas enquanto a thread principal acompanha o teclado.
5. Quando o usuário digita `/sair`, a conexão é encerrada, o servidor remove o cliente da lista e avisa os demais participantes.

Exemplo de mensagem recebida: `Ana: Olá, pessoal!`

Chat UDP multicast

Cada participante entra no grupo `230.0.0.1:12346` por meio de `MulticastSocket`. Ao enviar uma mensagem, o programa cria um datagrama endereçado ao grupo; os participantes inscritos podem recebê-lo. Não é necessário iniciar o `ServidorTCP` para essa versão. Multicast é envio a um grupo, diferentemente do broadcast IP para toda uma rede. Como o transporte é UDP, a entrega e a ordem não são garantidas.

Configuração do ambiente

É necessário um JDK com `java` e `javac` disponíveis no `PATH`. O projeto foi compilado e testado com JDK 21. Em PowerShell, verifique:

```
java -version
javac -version
```

Abra um terminal na pasta dos arquivos `.java` e compile:

```
javac -encoding UTF-8 ServidorTCP.java ClienteTCP.java ClienteMulticastUDP.java
```

O comando cria arquivos `.class` na mesma pasta. Salve e compile os fontes em UTF-8 para preservar os acentos.

Execução e teste

TCP

No primeiro terminal:

```
java ServidorTCP
```

Em dois ou mais terminais, na mesma pasta:

```
java ClienteTCP
```

Digite um apelido em cada cliente e envie mensagens. Confirme que aparecem no console do servidor e em todos os clientes. Digite `/sair` em um cliente e observe que os outros continuam conectados. Para acessar um servidor em outro computador, execute `java ClienteTCP ENDERECO_DO_SERVIDOR` e permita a porta TCP 12345 na rede.

UDP multicast

Em dois ou mais terminais, na mesma pasta:

```
java ClienteMulticastUDP
```

Digite os apelidos e envie mensagens. Confirme que uma mensagem enviada por um participante aparece no outro. Use `/sair` para encerrar. Em computadores diferentes, a rede e o firewall devem permitir multicast IPv4 e a porta UDP 12346.

Verificação realizada

Os três fontes compilaram com JDK 21. Em um teste local, dois processos `ClienteTCP` trocaram mensagens, o servidor mostrou o texto recebido e `/sair` encerrou a conexão. Dois processos `ClienteMulticastUDP` ingressaram no mesmo grupo, e a mensagem enviada por um foi recebida pelo outro. O teste entre computadores diferentes não foi realizado.

Respostas às questões teóricas

1. Comunicação síncrona e assíncrona

Na comunicação síncrona, o emissor espera uma resposta antes de continuar a etapa que depende dela. Exemplo: um sistema de vendas consulta o estoque e aguarda a confirmação antes de finalizar um pedido. Na comunicação assíncrona, o emissor envia a mensagem e segue com outras tarefas; o processamento ocorre depois. Exemplo: após uma compra, a loja publica um evento para que outro serviço envie o e-mail de confirmação.

2. Comparação entre TCP e UDP

O TCP estabelece conexão e oferece entrega confiável e ordenada, com retransmissão quando necessário. Ele é adequado ao chat desta atividade, pois perder mensagens ou alterar sua ordem prejudicaria a conversa. O UDP não estabelece conexão e não garante chegada, ordem ou ausência de duplicações; se essas garantias forem necessárias, a aplicação deve implementá-las. O UDP pode ser adequado a áudio ao vivo, em que dados recentes geralmente são mais úteis do que a retransmissão de um pacote antigo.

3. Uma thread por cliente no servidor TCP

A leitura de uma conexão pode ficar bloqueada enquanto o cliente não envia dados. Uma thread por conexão permite que o servidor espere esse cliente sem interromper a aceitação de novas conexões e o atendimento dos demais. Assim, vários clientes participam simultaneamente. Para uma quantidade muito grande de conexões, essa abordagem pode exigir mais recursos do sistema.

4. Necessidade de `synchronized`

Várias threads adicionam, removem e percorrem a lista de clientes. Se essas operações ocorrerem ao mesmo tempo sem coordenação, o percurso pode falhar ou o envio pode ficar inconsistente. Os blocos `synchronized (clientes)` protegem a lista com exclusão mútua, inclusive durante a retransmissão das mensagens.

5. Uso de middleware

Middleware é uma camada de software que oferece serviços comuns entre componentes distribuídos, como localização de serviços, troca de mensagens, serialização, autenticação e tratamento de falhas. Com uma interface conhecida para essas tarefas, cada componente precisa implementar menos detalhes de comunicação e pode se concentrar na regra de negócio.

6. Cenário adequado para multicast

Em um laboratório, um professor pode transmitir o mesmo aviso em tempo real a dezenas de computadores. Com multicast, ele envia o datagrama ao endereço do grupo e os participantes inscritos podem recebê-lo, sem que a aplicação envie uma cópia individual para cada destinatário. A rede precisa oferecer suporte ao multicast, e a aplicação deve aceitar que UDP não garante a entrega.